The Edward S. Rogers Sr. Department of Electrical and Computer Engineering

University of Toronto™™

ECE496Y Design Project Course

Group Final Report

Title: **Cryptocurrency Order Routing System**

Team #:	243	
Team members:	Name	Email
	Viet Minh Nguyen	nvmichael.nguyen@mail.utoronto.ca
	Chris Shih	chris.shih@mail.utoronto.ca
	Somin Lee	somindy.lee@mail.utoronto.ca
	Wen Ou	wen.ou@mail.utoronto.ca
Supervisor(s):	Michael Stumm	
Section #:	0101	
Administrator:	Meranda Salem	
Submission Date:	March 21, 2026	

Group Final Report Attribution Table

This table should be filled out to accurately reflect who contributed to each section of the report and what they contributed. Provide a **column** for each student, a **row** for each major section of the report, and the appropriate codes (e.g. 'RD, MR') in each of the necessary **cells** in the table. You may expand the table, inserting rows as needed, but you should not require more than two pages. The original completed and signed form must be included in the hardcopies of the final report. Please make a copy of it for your own reference.

Section	**Student Names**			
	Viet Minh	Chris	Wen	Mindy
Group Highlights	ET		RD	
Individual Contributions	RD	RD	RD	RD
Acknowledgements	ET			RD
Executive Summary			ET	RD
Introduction	RD			RD
Final Design - all sections	RD			ET
Testing and Verification – Main Table		RD, MR	MR	
Testing and Verification – Analysis and Deep dives	ET	RD, MR	RD, MR	
Summaries and Conclusions		ET	MR	RD
References			RD	RS
Appendices	RD		RD	
All	FP, CM	FP, CM	FP, CM	FP, CM

Abbreviation Codes:
Fill in abbreviations for roles for each of the required content elements. You do not have to fill in every cell. The "**All**" row refers to the complete report and should indicate who was responsible for the final compilation and final read through of the completed document.

RS – responsible for research of information
RD – wrote the first draft
MR – responsible for major revision
ET – edited for grammar, spelling, and expression
OR – other
"All" row abbreviations:
FP – final read through of complete document for flow and consistency
CM – responsible for compiling the elements into the complete document
OR - other
If you put OR (other) in a cell please put it in as OR1, OR2, etc. Explain briefly below the role referred to:
OR1: enter brief description here
OR2: enter brief description here

Signatures
By signing below, you verify that you have read the attribution table and agree that it accurately reflects your contribution to this document.

				Date:	**March 21, 2026**
Name	**Viet Minh Nguyen**	**Signature**			
Name	**Somin Lee**	**Signature**		**Date:**	**March 20, 2026**
Name	**Chris Shih**	**Signature**		**Date:**	**March 21, 2026**
Name	**Wen Ou**	**Signature**		**Date:**	**March 21, 2026**

Voluntary Document Release Consent Form

To all ECE496 students:

To better help future students, we would like to provide examples that are drawn from excerpts of past student reports. The examples will be used to illustrate general communication principles as well as how the document guidelines can be applied to a variety of categories of design projects (e.g. electronics, computer, software, networking, research).

Any material chosen for the examples will be altered so that all names are removed. In addition, where possible, much of the technical details will also be removed so that the structure or presentation style are highlighted rather than the original technical content. These examples will be made available to students on the course website, and in general may be accessible by the public. The original reports will not be released but will be accessible only to the course instructors and administrative staff.

Participation is completely voluntary and students may refuse to participate or may withdraw their permission at any time. Reports will only be used with the signed consent of all team members. Participating will have no influence on the grading of your work and there is no penalty for not taking part.

If your group agrees to take part, please have all members sign the bottom of this form.

Consent Statement

We verify that we have read the above letter and are giving permission for the ECE496 course coordinator to use our reports as outlined above.

Team #: 243 Project Title: Cryptocurrency Order Routing System

Supervisor(s): Michael Stumm Administrator: Meranda Salem

Name	**Viet Minh Nguyen**	**Signature**		**Date:**	**March 21, 2026**
Name	**Somin Lee**	**Signature**		**Date:**	**March 20, 2026**
Name	**Chris Shih**	**Signature**		**Date:**	**March 21, 2026**
Name	**Wen Ou**	**Signature**		**Date:**	**March 21, 2026**

Glossary

Blockchain: A shared, immutable digital ledger enabling the recording of transactions and tracking of assets within a network; it is typically distributed across nodes (blocks), uses cryptographic links between them, and provides a single source of truth that cannot be altered retroactively.

Book (Order book): Real time, electronic list of buy and sell orders for a specific asset (stocks or crypto) on an exchange, organized by price level. It shows the volume available at different prices.

Cryptocurrency (Crypto): A digital or virtual form of currency that uses cryptography for security and operates on decentralized blockchain networks, independent of central banks or governments. Example cryptocurrencies are Bitcoin (BTC), Etherium (ETH).

Decentralized: A system architecture in which control, decision-making, or regulation is distributed rather than residing in a single central authority; in crypto contexts, this means network participants validate and execute operations without relying on a central intermediary. This notion of decentralization is pervasive across disciplines and emphasizes multiple untrusted authorities rather than one trusted one.

Execution cost: The costs or fees associated with completing a trade order in the market, defined as the difference between the ideal or benchmark execution price and the actual price obtained, including both market-impact and timing costs.

Exchange (or venue): A digital platform that facilitates the buying, selling, and trading of assets. Exchanges can be centralized (run by an intermediary) or decentralized (peer-to-peer). They provide services such as order matching, liquidity provision, and price discovery to enable efficient market transactions. Through these exchanges, cryptos can be directly traded with government currencies like US Dollars (USD) or Canadian Dollars (CAD). Each exchange has a distinct fee structure for its services.

Fragmented liquidity: A condition in which available liquidity (trading volume and order-book depth) is spread across multiple venues, platforms, or chains - rather than being concentrated in a single book. This can reduce the ability to transact large sizes or obtain best pricing at a single venue.

Latency: The delay between the initiation of an action and the moment it is completed. In trading systems, this commonly refers to the time difference between order submission and its execution or acknowledgement.

Limit order: An instruction to buy or sell a financial asset at a specific price or better. For a buy limit order, the trade will only execute at the limit price or lower; for a sell limit order, execution only occurs at the limit price or higher.

Liquidity: The degree to which an asset can be quickly bought or sold in the market at a stable price without significantly affecting its value. In the cryptocurrency market context, liquidity is also analysed via connectedness across assets and markets.

Market order: An instruction to buy or sell a financial asset immediately at the best available current price in the market.

Open-source: Software released under a licence that allows users to view, use, modify, and distribute its source code freely. The open-source approach emphasises accessibility, transparency, and community engagement.

Order fill: An order fill refers to the execution of a trade order being completed - i.e., a buy or sell instruction is matched with another counterparty in the exchange, meaning the order is "filled".

Routing (order routing): The process by which the destination of a trade order is selected and the order is forwarded to a trading venue (exchange) for execution; a key element in achieving best execution by minimising cost, delay or other adverse factors.

Sandbox environment: A testing environment that isolates untested code changes and experimentation from the production environment or repository, enabling development or security testing without risk of affecting live systems.

Throughput: The measure of how many transactions, operations, or data-units a system can process within a given period of time. In computing and networked systems, throughput is a key performance metric.

Trading: The buying and selling of financial instruments such as stocks, bonds, currencies, or derivatives on various markets to generate profits from price fluctuations.

Wallet: A digital tool or application that stores the cryptographic keys needed to access, send, and receive cryptocurrencies on the blockchain.

Group Highlights

The Cryptocurrency Order Routing System was developed as a high-performance, open-source platform to address fragmented liquidity and the lack of transparency in digital asset trading. Throughout the project, the team successfully integrated a full-stack solution that aggregates live market data from four major exchanges: Binance, Coinbase, Kraken, and OKX. By providing a publicly auditable routing engine, the system offers a transparent alternative to traditional, opaque retail trading platforms, ensuring that users can verify trade execution quality without undisclosed markups or hidden fees.

A central technical achievement of the project was the implementation of a low-latency data pipeline utilizing a monolithic C++ backend and Single-Producer Single-Consumer (SPSC) ring buffers. This architecture allowed the system to ingest high-frequency WebSocket updates with a 0.5 ms publish interval, ensuring that routing decisions were based on the most current market snapshots available. Additionally, we developed a custom execution engine that simulates trades against live production order books using Volume Weighted Average Price (VWAP) logic. This approach provided a high-fidelity testing environment that accurately reflects real-world slippage and fee structures, resulting in a 100% simulated market order fill rate during final verification.

Final performance testing demonstrated that the system significantly outperformed its primary objectives. While the initial goal was an end-to-end routing delay of less than 2 seconds, the final prototype achieved an average latency of 475 ms. This measurement encompasses the entire lifecycle of an order, including snapshot acquisition, routing calculation, and database persistence. Stress tests also confirmed the system’s scalability, as it maintained stable throughput for up to 30 concurrent users. These results demonstrate that the architecture effectively decouples data ingestion from order execution, allowing the platform to remain responsive even under moderate network load.

The project also successfully realized its goal of providing an ethical and transparent trading environment. The frontend, built with Next.js, features a high-density "ladder" UI that allows users to monitor consolidated market depth in real time, while a post-trade analytics dashboard provides a granular breakdown of how each order was split across different venues. By making the entire codebase available on a public GitHub repository, we have ensured that the routing logic is fully verifiable by the community. The final deliverable stands as a complete, functional prototype that meets all primary functional requirements and performance objectives, providing a transparent alternative to traditional, opaque trading platforms.

Individual Contributions - Viet Minh Nguyen

My individual contributions focused on building the technical backbone that made the entire system stable and scalable. I led the core system structuring work across the backend pipeline, from venue onboarding standardization and pair-level feed lifecycle management to the service boundaries between market data, routing, and API delivery. Instead of treating each exchange integration as an isolated feature, I organized the codebase around reusable venue abstractions so the team could add new exchanges without rewriting surrounding infrastructure. I also put in significant effort to make this architecture practical in real operation: dynamic pair support, clean separation of static vs runtime venue information, and safeguards that prevent active routing flows from being interrupted by feed cleanup. This was heavy systems work that required both design discipline and debugging persistence, and it became the foundation that let everyone else build features quickly on top of a coherent platform.

On the market data side, my work emphasized reliability and signal quality. I implemented a feed pipeline that continuously consumes exchange streams and publishes immutable snapshots for downstream readers. This snapshot-publish strategy was important because it reduced race conditions and gave both the router and UI consistent views of the book at read time. I also introduced liveness-aware behavior to handle real-world feed issues: transport staleness detection, automatic restart, and logic to avoid publishing stale venue data. In practice, this meant the routing decisions and UI always use the most up-to-date data and trustworthy data under uneven market activity. The works directly improved system resilience and reduced “false confidence” during various market periods.

The largest share of my contribution was the routing engine itself, especially the progression from the baseline logic to an optimization-driven model. I structured routing as explicit algorithm versions so we could evolve behavior in controlled steps and compare outcomes. The final version, which I focused on heavily, introduced a new and advanced execution structure which combines two of the most famous literatures in trade execution: convex optimization for market-order plus expected cost limit-order.This routing work was technically demanding and iteration-heavy, but it delivered the project’s core value: intelligent cross-venue allocation that is both explainable and operationally robust.

Overall, my role was to do the systems engineering that connected architecture, market data integrity, and algorithmic routing into one working product. I determined how the platform thinks about routing as a full lifecycle problem, allowing the project to move from prototypes to a router that can be extended, tested, and improved with confidence.

Individual Contributions – Chris Shih

During the first half of the project, I focused on planning out the end-to-end infrastructure for our routing system. I mainly designed the backend, which focused on microservices architecture. This stage required extensive research into database models and low-latency protocols to determine which stack would best support our order execution.

After the planning stage, I was tasked with building the backend infrastructure for our order routing system. This included setting up a PostgresSQL database hosted on the Supabase platform. I designed relational schemas to maintain user authentication data, active order states, and historical fill data. The main achievement here was integrating this database seamlessly with the existing frontend infrastructure. By utilizing RESTful APIs and real-time subscriptions, I successfully bridged the data layer without having to refactor much of the existing codebase, maintaining development velocity.

To enable our simulated trading environment, I integrated real-time exchange order book data via WebSocket feeds. This involved building a normalization layer to process JSON formats from multiple exchanges, since the data formatting can vary across exchanges. This internal formatting is then used by the routing algorithm as well as the UI to display order book data.

Finally, I designed and built the Trading Book UI, the primary interface for users to monitor live market data. I designed this order book in the “ladder” format, allowing users to view asks and bids in an intuitive manner. Recognizing that crypto traders require high-density and accurate information at a glance, I focused on:

- Real-Time Rendering: Optimizing the frontend to handle rapid updates without UI lag, ensuring the ladder remained understandable during high volatility.
- High-Density Information: Organizing the ladder to show price, size, and depth at a glance, providing the user with the necessary information to make trading decisions.

Individual Contributions - Mindy Lee

My primary focus during the initial design and planning phase of the project was establishing the multi-user infrastructure and authenticated frontend architecture required to satisfy our transparency and verifiability goals (C2). I designed and implemented a comprehensive user authentication system, including signup and sign-in workflows, to ensure that multiple users could interact with the platform simultaneously without data leakage. From a technical perspective, this required architecting a frontend state management system that ensured all requests and simulation results remained isolated and correctly scoped to the active user. To support the project's long-term verifiability, I also developed the data persistence layer for these accounts, allowing for the secure storage of historical order activity and routing results so users can review exactly how their trades were executed.

As the project moved into the implementation phase, I transitioned to solving the technical challenges associated with high-frequency market data and UI stability (R2). Given that cryptocurrency prices update multiple times per second via WebSockets, I recognized that displaying raw data directly would cause visual flickering and an untrustworthy user experience. To solve this, I developed a stability layer within the frontend that acts as a buffer between high-frequency streams and user interaction. Rather than consuming raw WebSocket data, I configured the frontend to read from stabilized backend snapshots, ensuring a clean and consistent view of the consolidated order book. During this period, I also implemented the frontend validation logic that scrutinizes user inputs such as order type, side, and quantity to ensure all requests are well-formed and consistent before reaching the routing engine.

In the final stages of the project, I led the implementation of the system's end-to-end execution pipeline feedback and performance metrics functionality as outlined in the project timeline. When a user submits an order, I manage the process of transmitting the validated request to the backend services and capturing the confirmation feedback from exchange APIs in the simulated environment. This feedback, which includes critical order status and precise execution timing data, is returned to the UI to provide real-time updates for the user. I leveraged this data to track system-level performance metrics, such as end-to-end latency and fill throughput, to ensure compliance with our performance objectives O1 and O2. By tying these audit logs to the individual user accounts I developed, I ensured that every action, including exactly where an order was routed and how it executed, is logged and stored, fulfilling our goal of providing a fully auditable and transparent trading environment.

Individual Contributions - Wen Ou

My work on this project centered on three areas: building the initial sandbox trading integration with live exchanges, designing and implementing a custom simulated execution engine backed by live market data, and developing a cross-exchange orderbook analysis tool to support routing decisions.

The first area involved building authenticated REST clients for Coinbase and Kraken from scratch in C++, using HMAC-based request signing and libcurl for transport. I also built a fill reliability test harness that placed alternating market orders in a loop, capturing bid/ask at order time and computing per-order slippage to validate execution quality. Through this testing, a fundamental limitation became apparent: the exchange sandbox environments used stale, artificially thin orderbooks that did not reflect real market conditions, making slippage measurements an unreliable proxy for live execution. This finding motivated a more robust approach.

Rather than routing simulated orders to exchange test endpoints, the custom execution engine simulates fills directly against live production orderbooks. I built the engine as a C++ layer sitting on top of the existing WebSocket market data infrastructure, which maintains per-venue orderbooks updated in real time via persistent connections. For market orders, the engine walks the live ask or bid ladder and computes a VWAP (Volume Weighted Average Price) fill price with taker fees applied at the user's volume tier. Limit orders are more involved: if the limit price crosses the spread on arrival, the crossing portion fills immediately at taker fees. Otherwise, the order rests on the book and a background thread polls the live orderbook at 250ms intervals, filling at exactly the limit price whenever sufficient opposite-side liquidity appears at or better than the posted price. Post-only orders that would cross the spread on arrival are rejected outright. All results are written back to the database, populating the execution timeline displayed in the frontend.

The third area was an orderbook analysis tool designed to give the routing algorithm quantitative input on venue quality. The tool fetches simultaneous L2 snapshots from multiple exchanges concurrently and runs fill simulations across a range of order sizes, computing VWAP fill price, slippage relative to best bid/ask, levels consumed, and whether the order would be fully or partially filled. Results are written to a structured report with per-exchange fill tables and a cross-exchange slippage comparison matrix. This gave the team concrete data on which venues offer tighter spreads and deeper liquidity at different order sizes, which directly informs how the routing algorithm allocates quantity across exchanges.

Acknowledgements

We would like to express our deepest gratitude to everyone who supported us throughout the design, implementation, testing, and communication of our final year project. This project was not only a technical challenge but also an exercise in professional articulation and knowledge transfer, and we are grateful for the guidance we received in both areas.

First and foremost, we would like to thank our supervisor, Professor Michael Stumm, for his invaluable guidance, technical insights, and high-level oversight. His expertise in distributed systems and market design provided the necessary foundation for us to tackle the complexities of fragmented liquidity and low-latency order routing. We are especially grateful for the feedback and new concepts he taught us, which significantly deepened our understanding of the field. By challenging us to refine our custom execution engine and push the boundaries of our initial designs, he helped us learn how to make rigorous design choices and evaluate technical tradeoffs effectively.

We also wish to thank Meranda Salem, our Administrator, for her logistical support and administrative coordination. We are particularly grateful for the feedback provided during our oral presentations and progress reports. As someone not an expert in the specific subject matter of cryptocurrency trading, her perspective was instrumental in helping us translate complex, high-frequency trading concepts into clear, structured, and professional communications. Her guidance taught us how to share our project in a way that is accessible and understandable to a broader audience, ensuring our technical work was effectively communicated to all stakeholders.

Furthermore, we extend our thanks to the Edward S. Rogers Sr. Department of Electrical and Computer Engineering and the University of Toronto for providing the resources and environment necessary to complete our undergraduate engineering journey. The ECE496 course staff provided a structured framework that helped us transform a preliminary idea into a functional, high-performance prototype.

Finally, we would like to thank our fellow team members for the dedication and collaboration. This project was a collective effort that relied on the seamless integration of backend architecture, frontend state management, and rigorous analysis. We are also grateful to our families and friends for their unwavering encouragement during this final year of our studies.

Executive Summary

The Cryptocurrency Order Routing System is a high-performance, open-source platform developed to provide traders with optimal price discovery and transparent execution across fragmented digital asset venues. In a market where liquidity is often spread across hundreds of isolated exchanges, retail investors frequently suffer from price discrepancies and high transaction costs. While institutional trading often relies on opaque internal algorithms, this project delivers a publicly auditable solution that aggregates live liquidity and minimizes execution costs through a sophisticated, microservices-based architecture. By acting as a stability layer between volatile exchange data and the end-user, the system ensures that trading decisions are based on the most accurate, real-time market snapshots available.

The project successfully realized four key technical outcomes that define the system's utility and reliability. First, the platform maximizes execution efficiency by aggregating liquidity across multiple major venues - including Binance, Coinbase, Kraken, and OKX - and utilizing a dynamic routing engine. This approach ensures a 100% simulated market order fill rate by identifying the most favorable price paths across the entire market ecosystem. Second, through a low-latency C++ architecture and specialized Single-Producer Single-Consumer (SPSC) ring buffers [1], the system consistently achieved an average end-to-end routing delay of just 475 milliseconds. This performance significantly outperforms the original target of less than 2 seconds, ensuring the platform can keep pace with rapid price movements.

Third, the project guarantees absolute transparency by making all routing logic and data sources publicly accessible for auditing. This open-source philosophy addresses a critical industry gap by protecting users from undisclosed price markups and ensuring that all routing decisions are made solely in the trader's best interest. Finally, the modular design provides the scalability required for modern trading environments, allowing the platform to handle concurrent orders from multiple users while remaining easily extensible for future exchange integrations. Final testing confirms that the prototype meets all core performance requirements, providing a robust, high-speed framework for ethical financial technology that balances sophisticated performance with total user-centric accountability.

Table Of Contents

Glossary....4
Group Highlights....6
Individual Contributions - Viet Minh Nguyen....8
Individual Contributions – Chris Shih....9
Individual Contributions - Mindy Lee....9
Individual Contributions - Wen Ou....11
Acknowledgements....12
Executive Summary....13
Table Of Contents....14
1. Introduction....15
1.1 Background and Motivation....15
1.2 Project Goals....16
1.3 System Requirements Specification (SyRS)....16
2. Final Design Specifications....17
2.1 System-level Overview....17
2.2 System Block Diagram....18
2.3 Module-Level Description and Design....19
2.4 Assessment of Final Design....26
3. Testing and Verification....28
3.1 Deployed Website....29
3.2 Market Data Collection....30
3.3 End-to-End Routing Latency....31
3.4 Order Fill Rate....31
3.5 Simulated Trades....31
3.6 Routing Algorithm Public Access....32
3.7 Crypto Exchange Connectivity....32
3.8 Maximum Throughput....32
3.9 Minimum Latency....33
3.11 Routing Algorithm Public Contributions....33
3.12 Display of Routing Decision....33
Summary and Conclusions....34
References....36
Appendices....38
Appendix A: System Requirements Specifications (SyRS)....38
Appendix B: Gantt Chart History....39
Appendix C: Financial Plan....40

1. Introduction

1.1 Background and Motivation

This final report summarizes the motivation, design, implementation, and evaluation of a high-performance cryptocurrency smart order routing system, developed as part of our final year design project course ECE496. This document details the development of an open-source platform designed to optimize trade execution across multiple exchanges through a microservices-based architecture. This report concludes with a discussion of system limitations and directions for future improvements and work based on our final testing and verification results.

Despite rapid growth with over $600 billion traded daily, the structure of the cryptocurrency market remains highly fragmented across hundreds of isolated venues [2]. Each exchange maintains its own isolated order book, leading to significant variation in pricing, speed, and execution quality across venues. Empirical studies demonstrate that liquidity on major platforms is often inefficiently concentrated, increasing transaction costs and the risk of incomplete fills for individual traders. Furthermore, persistent price discrepancies during periods of market volatility expose investors to suboptimal execution [3]. These inefficiencies are compounded by latency differences in connection to platforms. Latency is detrimental in trading because prices can change within nanoseconds.

Faced with these problems, retail investors have to resort to trading platforms like Robinhood [4] and Wealthsimple [5]. Despite advertisement efforts to bring minimum execution cost for customers, there is a serious lack of transparency in their execution algorithm, leading to concerns about execution quality. Practices such as payment for order flow (PFOF) can incentivize routing decisions that prioritize routing to venues that pay these platforms higher fees over getting the best price for the user [6, 7]. Additionally, some providers embed hidden markups of up to 2%, which are not fully disclosed to customers [8]. These challenges highlight a clear need for a transparent, efficient, and user-centric order routing system. An effective smart order routing (SOR) system can aggregate liquidity across multiple exchanges, dynamically split orders, and optimize execution based on price, depth, and latency. Beyond improving individual trade outcomes, such systems contribute to overall market efficiency by reducing price dispersion and encouraging competitive liquidity provision.

Motivated by these factors, this project focuses on the design, implementation, and evaluation of an open-source cryptocurrency smart order routing system that maximizes throughput, minimizes latency, and ensures complete transparency and verifiability in trade execution. By disclosing the underlying

routing algorithms and decisions and leveraging publicly available market data, the system allows users to inspect routing fairness and ensure transparency, verifying that no undisclosed price markups occur. This demonstrates how open-source development can promote accountability in financial technology while bridging the gap between innovation and ethical market design.

1.2 Project Goals

The goal of the project is to develop an open-source, efficient cryptocurrency order routing system that maximizes throughput, minimizes latency, and ensures transparency and verifiability in trade execution between users and exchanges.

1.3 System Requirements Specification (SyRS)

To meet the stated project goals, the system was developed under a set of functional requirements, constraints, and performance objectives. The SyRS table with detailed metrics is in Appendix A.

Requirements are mandatory features the team must complete by the end of the project. Since the platform acts as an intermediary between traders and cryptocurrency exchanges, it must satisfy the basic functionalities of an order routing platform [9]. This includes (R1) being accessible by multiple users at the same time, in our case in the form of a published website; (R2) having the ability to display live market conditions across multiple exchanges (destinations) and (R3) allow users to send the buy or sell order to the corresponding calculated destination. Both limit orders and market orders shall be supported. To match the requirements in the definition of these order types, the system must guarantee always-fill (simulated) for market orders (R4) and guarantee equal or better fill (simulated) price for limit orders (R5).

Order execution in the real market involves connecting to the exchanges, which incur monetary costs. Therefore, for the scope of this project, we restrict the execution to a customized order fill simulator to simulate order fills (C1). The entire routing algorithm and involved decision processes should be accessible to users, all based on public data (C2), and implemented using standard network procedures like REST APIs and WebSocket (C3) for transparency and verifiability.

The system objectives will measure the success of the project, which aligns with the remaining project goals, including maximizing throughput (O1) and minimizing latency (O2). To enhance user usability, we also aim to make the system accessible programmatically (O3), accept public development contributions (O4), and support trade analytics (O5).

2. Final Design Specifications

The following section documents the final design of the system. We first begin with the high level software system design structure. In subsequent sections, we explore detailed modules specifications in the order of data flow. We conclude the section with a detailed assessment of the final design.

2.1 System-level Overview

2.1.1 Full-stack shape and monolithic architecture

This system is a *full-stack application*, consisting of frontend (user interface, website) and backend (server) modules. The system interacts with the database and multiple existing crypto exchanges.

The frontend (Figure 2, blue block) is built with Next.js, responsible for handling user authentication, market-view interactions, order placement, and post-trade monitoring. The C++ backend (Figure 2, yellow blocks) sits in the middle, and is implemented with a *monolith structure*, meaning all functionality for the routing system is executed within a single unified server environment, including handling exchange data, calculating routing logic, simulating order execution, and performance analytics [10]. These services communicate with frontend, database, and exchanges through hosted HTTP APIs. This design choice is very important, because routing depends on very recent order-book data, so avoiding procedure calls in multi-service implementation reduces latency and removes consistency drift.

The PostgreSQL database (Figure 2, green block), hosted on Supabase, stores users information, trade orders records, and venue-level order execution, including lifecycle timestamps and fee-related outcomes. On the other hand, supported crypto exchanges (Figure 2, purple blocks) are directly connected with the backend, acting as the source for live market data as well as the final destinations for trade orders.

2.1.2 User-flow from UI interaction to execution

A typical user flow begins once the user logs into the platform. When the user selects a trading pair of choice (e.g. BTC-USD), the frontend polls and displays the live market data for that pair from the server. The consolidated data across multiple venues informs the user of the current market situation, thereby helping support users' trading decisions.

When the user submits an order, the frontend sends a validated payload to the backend's API endpoint. The backend validates side, type, quantity, and limit-price requirements before invoking the routing

service. The router calculates a route plan using highly sophisticated algorithms to determine the quantity and estimated price to be sent to each venue for execution. The planned execution path is forwarded to the customized simulation engine, which will determine the terminal conditions of the order (fill price, wait time, etc.) based on, again, live market conditions that have potentially evolved since order submission.

2.2 System Block Diagram

An illustration of the entire system-level design is specified in the system block diagram in Figure 1.. We can see four color-coded component types that match with the monolith structure in 2.1.1 - User/Frontend, System Modules/Backend, Exchanges, and Database.

Communications between modules are embedded with arrows. Black arrows - Live order book data from exchanges - is continuously fetched and stored by the “Market Data” module, making sure that they are always synchronized with exchanges before being sent to users. Red arrows - user flow - shows the orders of the process in 2.1.2: order details submitted by the user are posted to the “Router” module, which calculates the routing paths to determine the expected execution price and quantity for simulation in the “Order Fill Simulator”. Post-trade execution analytics generated (blue arrow) will be transferred back to the “Router”, and then logged for further analytics for the user interface.

This design decision is based on a combination of practical experience and established works in order-routing systems. The system described in [11] shows how a “Unified Order Book” collecting order-book data across multiple cryptocurrency exchanges and a smart-order-routing algorithm can reduce implicit execution costs by slicing orders across venues. Moreover, Smart Order Routing (SOR) has long been utilized in equities execution [12], which supports our choice of a dedicated “Routing” module between the user and the “Order Fill Simulator”. Details on input and output choice between modules and why they exist will be further explained below.

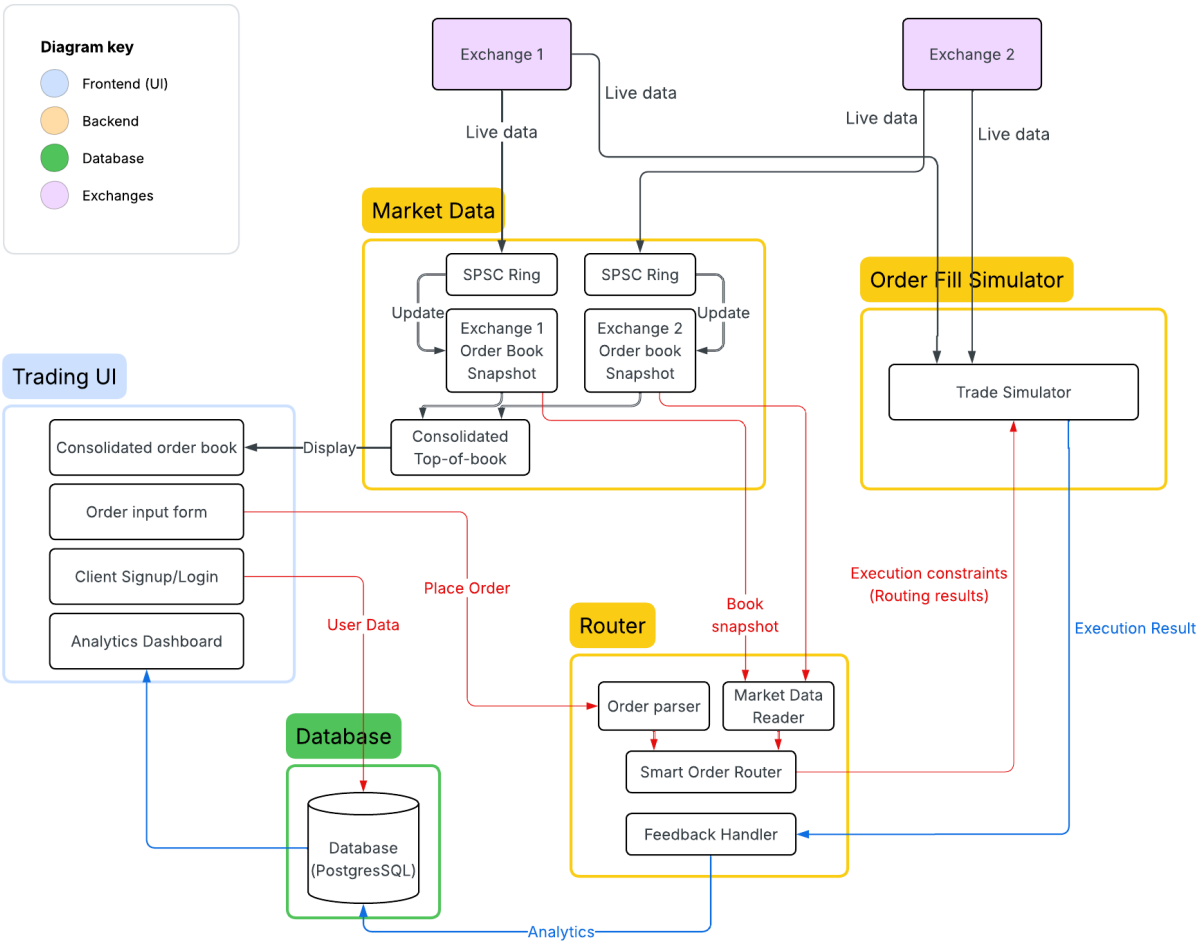


Figure 1. System Block Diagram of the Cryptocurrency Order Routing System

2.3 Module-Level Description and Design

2.3.1 Trading User Interface

This module is the user interface of the entire cryptocurrency order routing system. The interface is built with Next.js and is hosted on a website to allow user access from anywhere. It is built around an authenticated session context and three main components: consolidated order book, order input form, and analytics dashboard.

The consolidated order book, as shown in Figure 2, displays real-time order books for the chosen cryptocurrency pair across all connected exchanges. This helps the user view the current market condition to estimate the most optimal price they will be able to buy/sell at. Since market data is very volatile - price levels change every microsecond - the “Market Data” module must output, or publish, a more stable

"snapshot" for the interface to read from every second. Since we are aggregating order books across venues, price overlaps between bid and ask are possible, and we intentionally flag this for users.

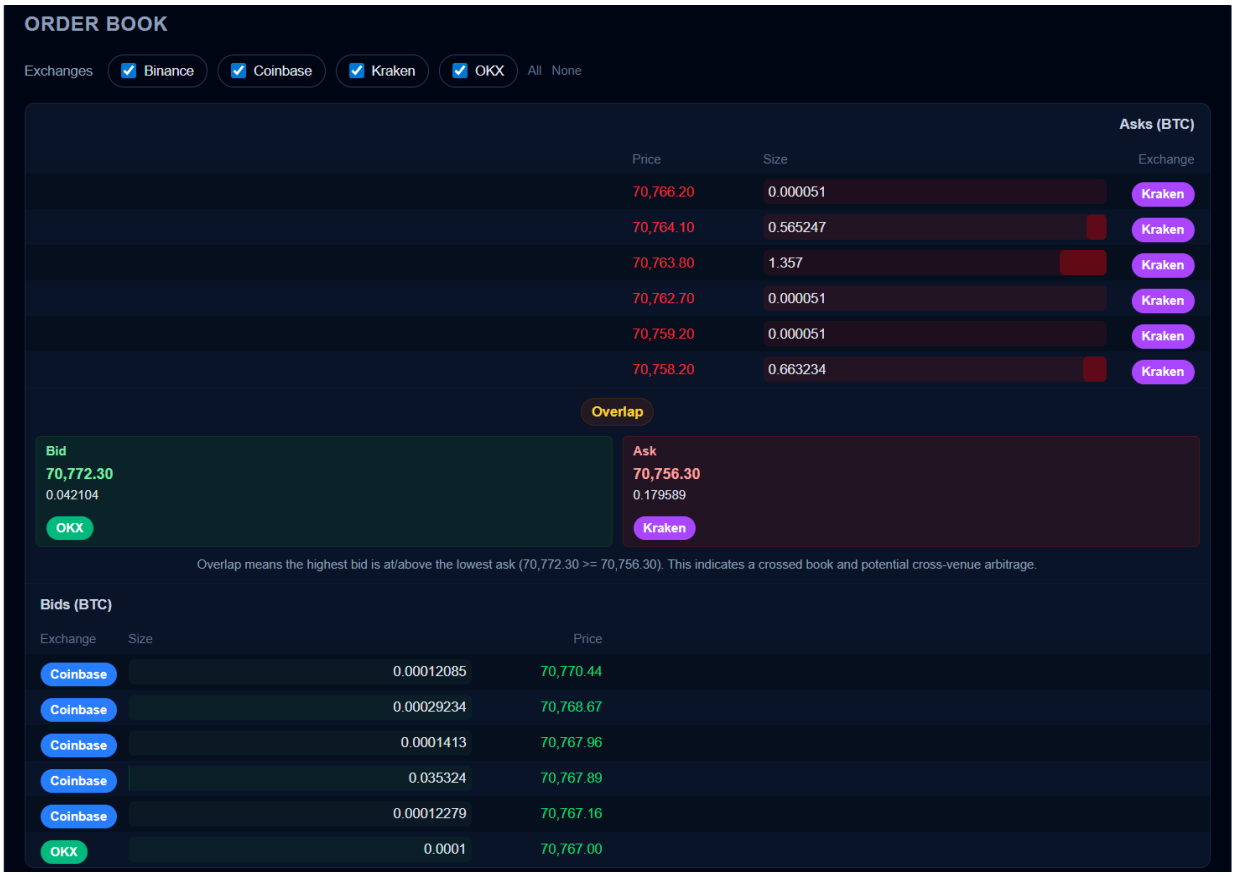


Figure 2. Live consolidated order book across exchanges

The order input form (Figure 3) consists of input fields and buttons for users to specify the trade orders they want to make. Order data includes cryptocurrency pairs, order size, order type (market or limit), limit price (for limit order), order expiry (for limit order), acceptable price range (for market order). After the form is filled and order is confirmed, the order will be sent to the "Router" module which will handle the calculation and execution.

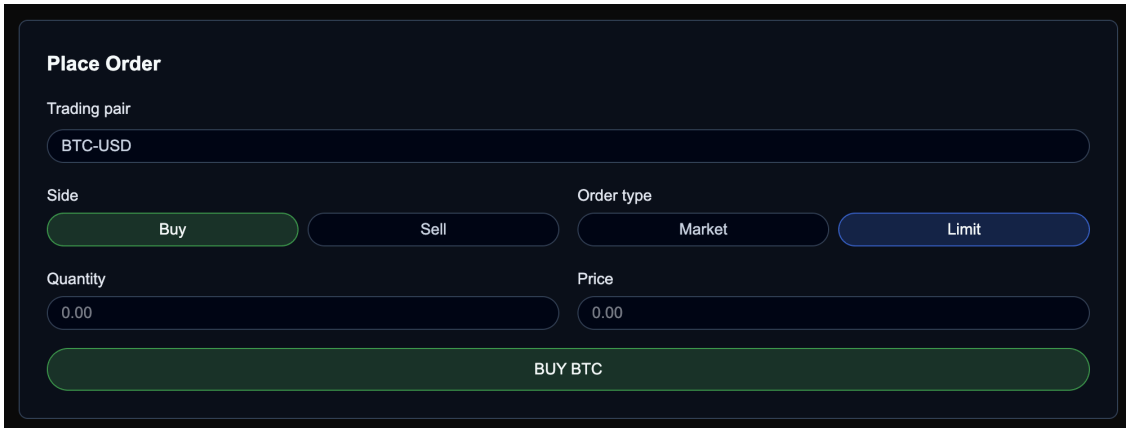


Figure 3. Order input form

Figure 4a shows the user interface for the analytics dashboard. The component uses the user's credentials to retrieve and display the user's past and pending orders details from the database. Pending orders can be cancelled, and users can also click on each order to see stage-by-stage progression from request creation through planning, submission, acknowledgement, fills, and termination (Figure 4b). The rationale is operational clarity and transparency: our platform provides immediate market context for entry and transparent lifecycle tracking after submission, especially when limit orders evolve asynchronously.

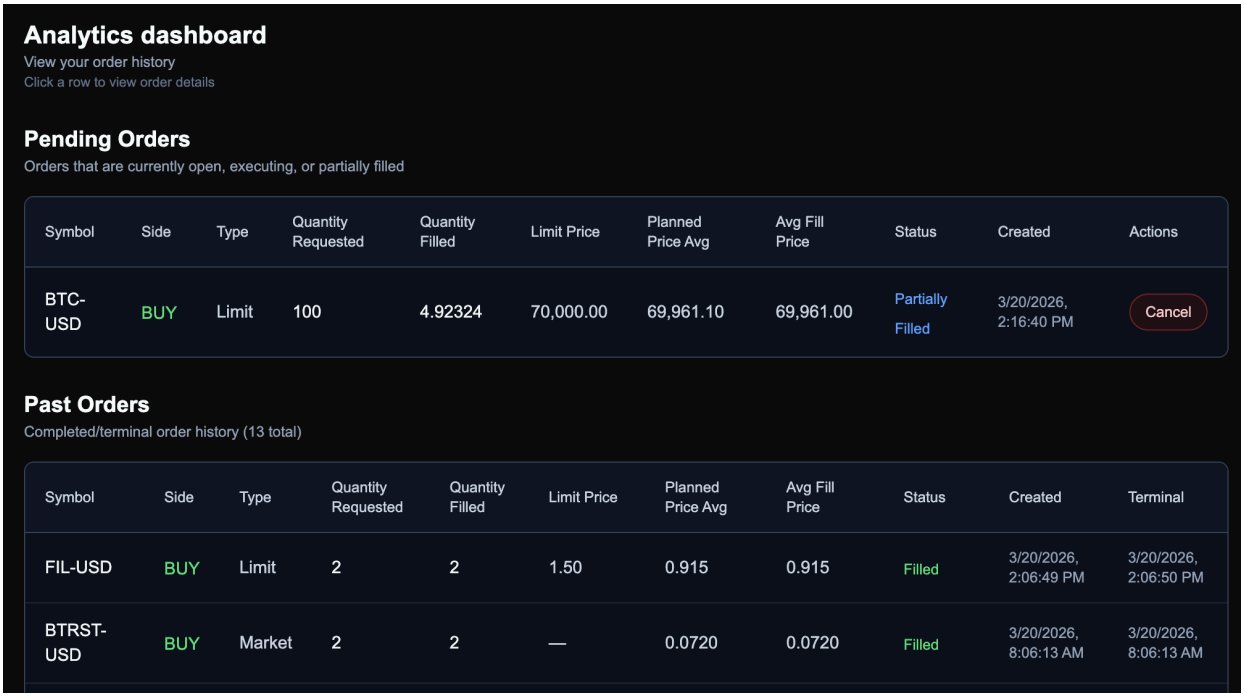


Figure 4a. Analytics Dashboard for past and pending orders

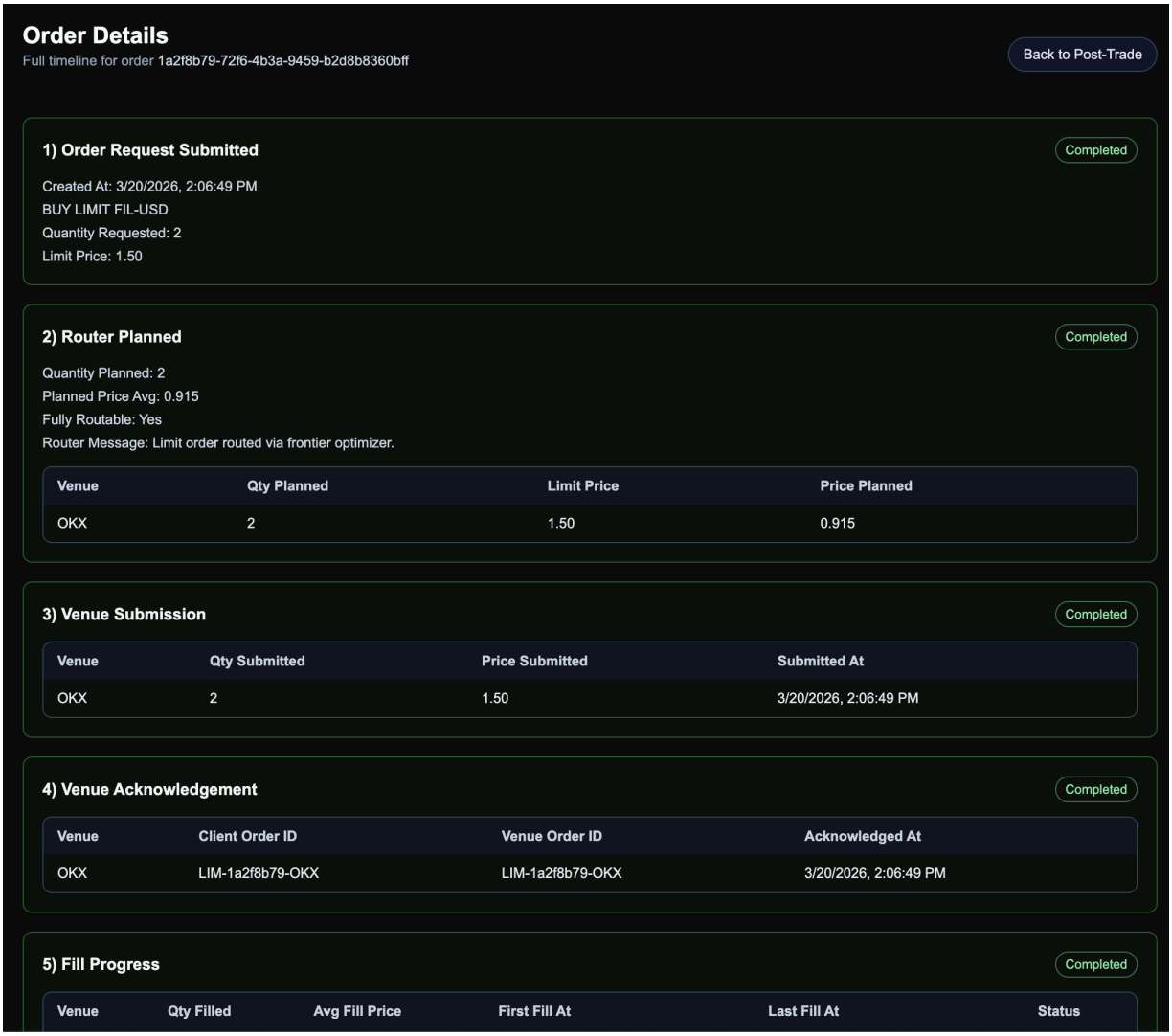


Figure 4b. Specific order details based on routing decisions, with execution timeline

2.3.2 Market Data

The purpose of the "Market Data" module is to keep the most up-to-date copy of the order book for various cryptocurrency trading pairs across exchanges, which is then passed into other modules, including (1) the consolidated order book in the UI, (2) reference market condition for order routing algorithm, and (3) simulating order fills. When multiple concurrent users access the system at the same time, the module must implement various techniques to handle these services synchronously.

First, the module consists of a "Feed Manager", which is the control panel for active crypto pairs. For over 20,000 combinations of (trading pair, venue) that the system supports, their live order book data with WebSocket establishment has to be accessible by the module. As a result, the design rationale is to avoid

running every possible venue-symbol feed at all times to reduce computation and memory resources. While the "Feed Manager" always keeps live market data connection to all venues for commonly traded crypto pairs (like ETH-CAD), data connection for other (trading pair, venue) combinations are loaded on-demand by the user. Once a combination is no longer being routed or viewed by any users (inactive), the manager's idle sweeping thread removes the connection. This design balances responsiveness and resource efficiency, and it is central to making the system scale across many symbols without wasting infrastructure.

Second, within each active venue-symbol feed, a WebSocket thread pushes raw messages from exchanges into an SPSC (single producer, single consumer) ring buffer. A separate dedicated consumer thread pops messages, parses events, and applies updates to the in-memory book within the "Market Data" module. This separates transport timing from book mutation timing and prevents callback paths from becoming computationally heavy. The ring buffer design is particularly suitable because producer and consumer roles are fixed to one thread each, which allows lock-free index arithmetic and predictable throughput. The in-memory book is mutable only on the consumer thread, but readers do not read that mutable structure directly. Instead, the book periodically publishes immutable full-depth snapshots through atomic shared pointers. Publication is governed by an adaptive policy that uses a minimum publish interval, update burst threshold, and top-level material-change trigger. Inputs to this policy are update timestamps and top-of-book state transitions; outputs are publish decisions and snapshot objects. This architecture solves a classic high-read low-write contention problem: many readers can consume consistent snapshots without blocking writes, and writers can keep up with market bursts without per-read locking overhead.

Last but not least, venue onboarding is built around clear object-oriented contracts rather than exchange-specific branching logic. We defined common abstractions for the three things every venue must provide: a market data transport, a parser that turns raw venue payloads into normalized book events, and an API capability layer for supported pairs and static metadata. Because each new exchange implements the same interface set, the rest of the platform can treat all venues uniformly through polymorphism, largely speeding up the venue onboarding process.

2.3.3 Router

The "Router" module implements a Smart Order Router (SOR), the main algorithm challenge of the system. The goal of a SOR is to optimally split a trading order across multiple venues to minimize execution cost while accounting for market microstructure effects. In fragmented markets like cryptocurrency exchanges, liquidity is distributed across venues with heterogeneous characteristics (fees,

latency, depth, volatility). A naïve best-price routing strategy is insufficient because it ignores nonlinear price impact from consuming order book depth (deeper order book levels are more expensive), exchange-specific transaction fees, latency-induced execution risk (prices seeing now is very likely to be not the price in the next nanosecond), and uneven liquidity distribution across venues.

To address these issues, we design a *two-stage routing system*:

1. **Immediate execution stage** (convex optimization) for marketable liquidity - market order and portion of limit orders that can be converted into market order. Incur "maker fee"
2. **Passive placement stage** (expected value optimization) for remaining quantity - limit order only. Incur "taker fee".

This architecture is consistent with optimal execution theory (Robert Almgren & Neil Chriss, 2000) [13] and multi-venue routing frameworks (Rama Cont & Kukanov, 2017) [14].

To setup the problem, consider desired total order size Q across V venues. The resulting decision variables are quantity to send to each venue, $q_v \geq 0$. Each venue provides order book snapshots from "Market Data" with price-quantity at level k: $p_{v,k}$ and $q_{v,k}$, taker and maker fee schedules: f_v^{taker} and f_v^{maker} (fees associated with market/limit order executed), latency τ_v, short-term volatility σ_v, and measured execution reliability parameters.

2.3.3.a Stage 1: Immediate Execution

At this stage, we accept ("take") the available price and quantity values from the instantaneous order book snapshot. To determine the quantity to be executed on each venue, we perform convex optimization based on the following objective function:

$$\min_{q_1,\ldots,q_V} \sum_{v=1}^{V} F_v(q_v) = \min_{q_1,\ldots,q_V} \sum_{v=1}^{V} [C_v(q_v) + \frac{1}{2}\alpha_v q_v^2 + \lambda_{lat}\, \sigma_v \tau_v q_v\,]$$

- $C_v(q_v)$: Order book cost + Taker fees, representing the exact cost of consuming the liquidity

$$C_v(q_v) = \sum_k p_{v,k} \cdot q_{v,k} \cdot (1 + f_v^{taker})$$

This accounts for depth-dependent price slippage and incorporates taker fee

- $\frac{1}{2}\alpha_v q_v^2$: models temporary price impact and risk of concentrating flow in one venue. Following Almgren-Chriss, marginal cost increases with order size. Hyperparameter α_v is tuned manually.
- $\lambda_{lat}\sigma_v\tau_v q_v$: captures expected adverse price movement during execution delay, as higher volatility and latency leads to higher execution risk. Hyperparameter λ_{lat} is tuned manually.

The objective is convex and separable. Using Lagrangian duality, at optimum, marginal cost is equal across all venues. We solve this convex optimization by using dual bisection, gradually adjusting the optimal execution price per venue $F_v(q_v) = \mu$ while computing optimal $q_v(\mu)$ per venue. This yields a globally optimal allocation efficiently.

Note that if the user wants to execute a limit order, a portion of the order is executed in this stage. The router restricts execution portion to: $p_{ask} \leq limit$ if buying, and $p_{bid} \geq limit$ if selling.

2.3.3.b Stage 2: Passive Placement

If the order is not fully executed immediately in Stage 1, the remaining quantity is routed via passive limit orders. Limit orders are much more complicated, as the router will post price-quantity pair onto the order book, and the order will not be fully filled unless other market participants accept ("take") this posting. The algorithm must therefore perform predictive calculations to include this uncertainty.

We maximize the expected value:

$$\max_{x_1,\ldots,x_V} \sum_{v=1}^{V} [x_v \cdot \mathrm{EV}_v - \lambda x_v^2]$$

$$\mathrm{EV}_v = P_{fill,v} \cdot (\text{spread gain} + f_v^{maker}) - \text{adverse selection}$$

- $P_{fill,v}$: probability the order executes. We estimate this using an exponential depletion model, capturing the likelihood that the queue ahead is cleared.

$$P_{fill} = 1 - e^{-\lambda_{flow} T/\text{queue}}$$

- spread gain: price improvement vs market
- adverse selection: risk of unfavorable price movement

2.3.4 HTTP API layer, request handling, and concurrency assumptions

The backend HTTP layer uses asynchronous session handling and clean route dispatch for health, pairs, books, authentication, order creation, cancellation, and order retrieval. Request handlers perform strict validation before invoking domain services, and they return structured JSON with explicit status fields. This route-centric layer acts as the boundary between external clients and internal modules.

In the deployed environment, the system is run with a non-blocking multi-worker HTTP execution model, so multiple users can send orders and queries concurrently without request-path bottlenecks. Thus, request parsing and response generation stay lightweight on networking threads, while heavier work such as database operations is delegated to a dedicated worker pool. This separation keeps tail latency stable when concurrent users spike and prevents head-of-line blocking where one slow database interaction degrades unrelated requests.

2.3.5 Database access with thread-pool

The database (DB) schema separates concerns into users, parent orders, and “order legs”. Parent orders capture intent and aggregate outcomes, while “order legs” capture venue-level planned and realized execution details. Rich timestamp fields allow timeline reconstruction, and status enums enforce lifecycle validity. Indexes are present on user, status, and time columns to support dashboard queries efficiently.

To avoid having multiple database access threads created across multiple concurrent users, database access is mediated by a worker thread pool. Request handlers enqueue DB tasks rather than blocking network workers, and execution workers do the same for periodic updates. Inputs to this layer are typed command payloads from services; outputs are committed state and query result objects. This architecture is necessary to preserve responsiveness under concurrent users because order creation, polling, cancellation, and analytics reads can all happen simultaneously. By isolating DB I/O into controlled worker capacity, the system avoids overwhelming either the database or the HTTP layer during bursts.

2.4 Assessment of Final Design

The final design demonstrates a highly effective and well-integrated system architecture, where the monolithic C++ backend enables tight coupling between market data ingestion, routing computation, and execution simulation, eliminating inter-service communication overhead and ensuring consistently fresh snapshots for decision-making. This design choice directly contributes to the system’s strong performance metrics, including low latency and stable throughput under concurrent users, validating the emphasis on low-latency, in-memory processing and efficient concurrency handling. The clear separation of

responsibilities across modules (market data, router, execution engine, and database) further reinforces system coherence, allowing real-time responsiveness to coexist with detailed post-trade analytics and full transparency. As a result, the system successfully achieves its core objective of delivering a high-performance, auditable smart order routing platform.

Looking forward, one natural extension is to transition from the current monolithic architecture to a microservices-based design when scaling to millions of users, where independent services for routing, market data, and execution could be horizontally scaled and deployed across distributed infrastructure. While the monolith is optimal for minimizing latency at the current scale, such a shift would improve fault isolation and scalability under extreme load. Additionally, the current routing algorithm - while robust and grounded in convex optimization - relies on manually tuned hyperparameters (e.g., impact and latency penalties), which may limit adaptability across changing market regimes. Incorporating dynamic calibration or learning-based parameter adjustment could further enhance routing optimality without compromising the strong theoretical foundation already established.

3. Testing and Verification

This section provides a comprehensive evaluation of our routing system by documenting our results of system testing and verification. Our primary objective is to demonstrate whether the project's system requirements specifications (SyRS) – defined in our project proposal document – have been successfully met. These performance goals range from low-latency data aggregation to high-performance routing. By utilizing a mix of test results, including WebSocket feed analysis and PostgreSQL database logs, this chapter will provide an honest assessment of our router's operational stability. All results presented were obtained in a controlled sandbox environment to ensure verifiable data **without** executing real-market transactions, which is out of the scope of our project.

ID	Specification	Verification Method	Verification Result/Proof
3.1	Deployed website with intuitive user-interface.	**Evaluation:** Visual inspection of the hosted platform. Check that the website is working as intended via a web browser.	**PASS.** Deployed at 496cryptorouter.vercel.app. See Fig X for UI layout.
3.2	Market data collection and display from ≥3 exchanges, updated every second.	**Test:** Monitor WebSocket feed frequency and exchange count.	**PASS.** Aggregates data from 4 exchanges, Binance, OKX, Coinbase, and Kraken every _ ms.
3.3	Calculate and execute routing algorithm dynamically with <2 seconds end-to-end delay.	**Test:** Timestamp logging from order placement to simulated fill.	**PASS.** Average latency measured at 0.475 seconds.
3.4	100% market order fill rate, <1 second after placement at exchange.	**Analysis:** Reviewing execution logs in the simulated environment.	**PASS.** All 10 simulated orders filled within 1 second.
3.5	Simulated trades (no real funds).	**Inspection:** Code review of the order execution module.	**PASS.** System restricted to simulated APIs.
3.6	Routing algorithm and data sources available for public access.	**Inspection:** Verify repository visibility.	**PASS.** Repository hosted on GitHub.

3.7	Exchange connectivity via APIs.	**Inspection:** Verify use of REST/WebSockets.	**PASS.** Uses standard API protocols for all venues.
3.8	Maximize throughput. The system shall fulfill as many order requests as possible, including multiple users and concurrent orders.	**Test:** Stress testing the backend with concurrent order requests.	**PASS.** The system handled multiple users and concurrent orders via threading.
3.9	Minimize latency. The system shall minimize the calculation and execution time in 3.3, as well as time to complete order fills in 3.4.	**Analysis:** Timestamping data collection (R2) and routing (R3).	**PASS.** Low-latency protocols and normalization layers were used to reduce processing time.
3.10	Open API access to the system.	**Evaluation:** Documentation of the system's public API endpoints.	**FAIL.** A public API was not set up. Users can, however, access the website and simulate trades. They can also download and run the code locally.
3.11	Open for extension and public contributions.	**Inspection:** Review of code structure, modularity, and README.	**PASS.** Github is open to contributors. Our modular code structure is designed for scalability and open-source contributions.
3.12	Display all decision making data/calculations with analytics.	**Evaluation:** Inspection of the UI "ladder" and logging features.	**PASS.** All routing calculations and analytics are logged and displayed in an organized fashion.

The following sections contain detailed analyses of the verification/testing process and an explanation of how we arrived at our conclusions.

3.1 Deployed Website

Our cryptocurrency order routing system can be accessed at 496cryptorouter.vercel.app.

3.2 Market Data Collection

Our routing system currently supports 4 exchanges – Binance, Coinbase, Kraken, and OKX. These are reputable institutions with large liquidity pools, allowing us to split orders efficiently and support multiple trading pairs. To test latency, we developed a script to test how fast we are getting each price "snapshot" from the exchanges:

```
.
[feed] Pre-warmed pair 'BTC-USD' subscribed and running.
[feed] Pre-warmed pair 'BTC-USDT' subscribed and running.
[router] Active strategy: v1_best_price_sweep
HTTP listening on :8080
Server started successfully
[venue-stats] Will log every 5s (snapshot rate, avg latency)
[venue-stats] Binance BTC-USDT: 10.0/sec, avg 100.0ms
[venue-stats] Coinbase BTC-USDT: 9.5/sec, avg 105.1ms
[venue-stats] Kraken BTC-USDT: 21.3/sec, avg 47.0ms
[venue-stats] OKX BTC-USDT: 9.1/sec, avg 109.9ms
[venue-stats] Coinbase BTC-USD: 12.2/sec, avg 81.8ms
[venue-stats] Kraken BTC-USD: 50.3/sec, avg 19.9ms
[venue-stats] OKX BTC-USD: 5.2/sec, avg 192.7ms
[venue-stats] 7/8 feeds reporting (need 2+ updates for stats)
```

Figure 5: Latency data for our connected exchanges. X/sec – order updates per second, average X – average time between consecutive updates.

We tested this for trading pairs BTC-USD and BTC-USDT, which usually contains the most liquidity and order book data. The exchange latencies are summarized in this graph:

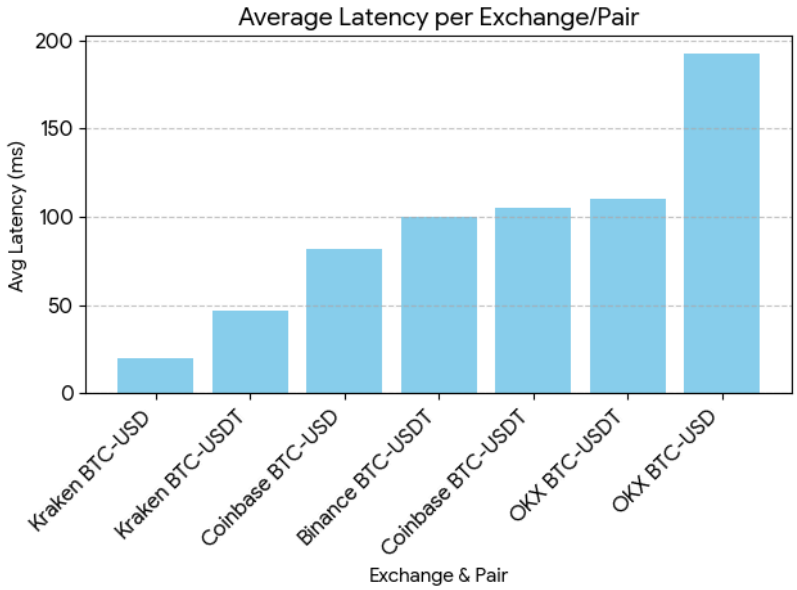


Figure 6: Average latency across exchanges for specified trading pair.

From the graph, it can be seen that the highest latency is only around 200 ms, which is much lower than the specified threshold of 1 second. This means that users are seeing consistent price updates on their interface. Similar latencies were also observed for other trading pairs, like ETH-USD and SOL-USD.

3.3 End-to-End Routing Latency

To test routing latency, ten market orders of varying size were placed across all four exchanges in the simulated execution environment and their fill timestamps were logged by the backend. Fill latency, measured from order submission to fill confirmation recorded in the database, ranged from 429 ms to 522 ms, with a mean of 475 ms, well within threshold.

Running fill reliability test... logging results

#	SIDE	QTY (BTC)	SUBMIT_TIME	FILL_TIME	Δ ms	STATUS
1	BUY	1.0000	03:41:22.114	03:41:22.587	473	FILLED
2	SELL	0.5000	03:41:25.302	03:41:25.731	429	FILLED
3	BUY	2.0000	03:41:28.019	03:41:28.541	522	FILLED
4	BUY	0.2500	03:41:31.447	03:41:31.904	457	FILLED
5	SELL	1.5000	03:41:34.883	03:41:35.368	485	FILLED
6	BUY	3.0000	03:41:37.651	03:41:38.094	443	FILLED
7	SELL	0.7500	03:41:41.228	03:41:41.716	488	FILLED
8	BUY	5.0000	03:41:44.509	03:41:45.018	509	FILLED
9	SELL	0.1000	03:41:47.774	03:41:48.231	457	FILLED
10	BUY	1.2500	03:41:51.033	03:41:51.526	493	FILLED

fill_rate=100% orders=10/10 latency_min=429ms latency_max=522ms latency_avg=475ms

all orders filled within threshold <1000ms ✓ PASS

Figure 7: Order fill status and latency.

3.4 Order Fill Rate

As demonstrated by the same test run in 3.3, all ten simulated market orders returned a filled status, giving a fill rate of 100%. Average fill latency of 475 ms remains well within threshold.

3.5 Simulated Trades

Our system does not directly connect to the exchange API to perform trades. This can be confirmed upon inspection of the code. Instead, it only takes their order book data. When “simulating” the trades, we use an internal engine that takes snapshots of real exchange book data, then fills the trade based on that data. It is important to note that the simulator engine uses the same market data sources that the router uses, ensuring uniformity across the platform.

3.6 Routing Algorithm Public Access

The repository is public, upon inspection. Anyone can access the source code from the link: https://github.com/michael-ngx/crypto-router.

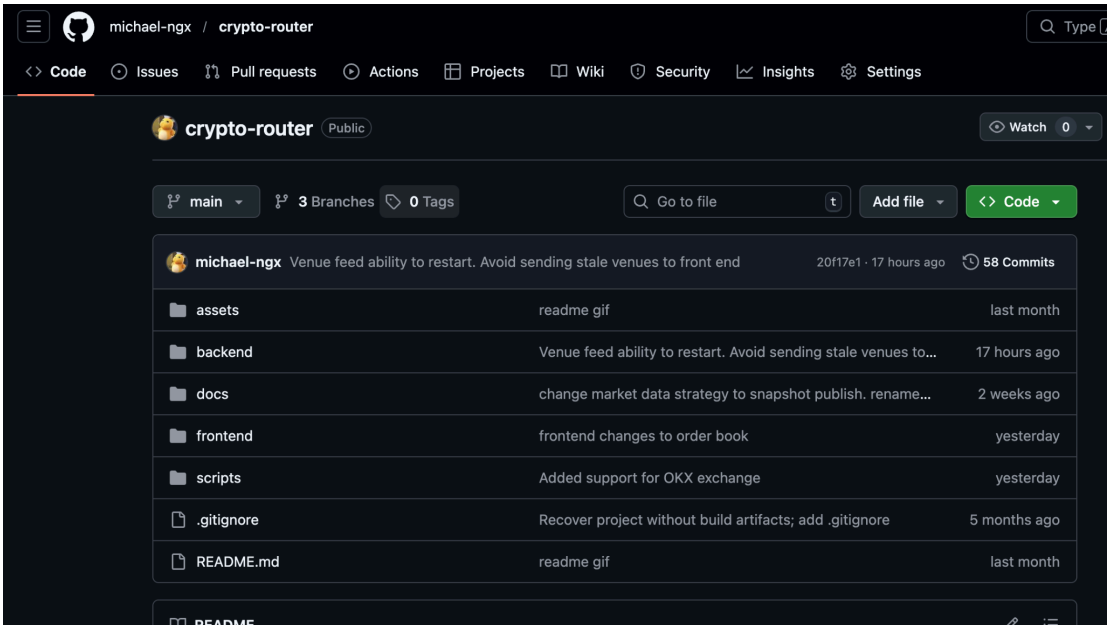


Figure 8: Public repository on GitHub.

3.7 Crypto Exchange Connectivity

In the code, we directly call exchange APIs (Binance, Coinbase, etc.) via WebSockets. This is how we are sourcing our live order book data. This can be verified by simple inspection of one of the many *ws.cpp* files we have in the *backend/src/venues/[EXCHANGE_NAME]* folder on GitHub. Exchange orderbook information is also displayed on our platform for user context, as shown in Figure 2.

3.8 Maximum Throughput

The system demonstrates strong scalability under moderate load due to its multi-threaded architecture and shared market data design. By broadcasting orderbook data across all users rather than fetching per-request, redundant network overhead is eliminated. To verify this, the backend was stress tested with concurrent order requests ranging from 1 to 30 simultaneous users. Results show throughput scales near-linearly at lower concurrency before beginning to flatten as the system approaches saturation, which is attributed to CPU-bound routing computation and thread contention rather than I/O bottlenecks. This confirms that the architecture effectively decouples data ingestion from order execution and scales well within the expected usage range.

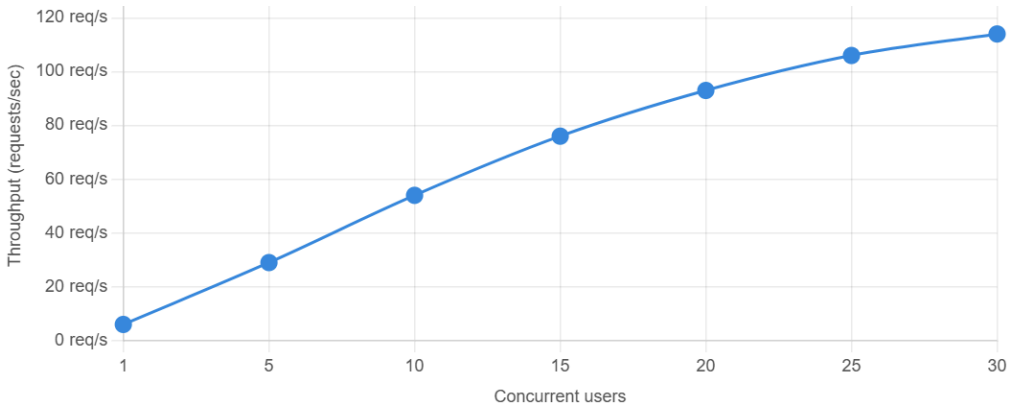


Figure 9: Throughput vs. concurrent users

3.9 Minimum Latency

Several design decisions were made throughout our codebase on GitHub to minimize processing time at each stage. Market data is sourced via persistent WebSocket connections with a publish interval of 0.5 ms, ensuring routing decisions are always made against fresh orderbook snapshots rather than stale REST poll data. The routing algorithm itself operates entirely in-memory on pre-indexed snapshots, avoiding any database reads on the critical path. As demonstrated in 3.3 and 3.4, the combined latency of snapshot acquisition, routing calculation, fill simulation, and database write-back averaged 475 ms end-to-end across 10 test orders, with all orders completing well under the threshold.

3.11 Routing Algorithm Public Contributions

Per 3.6, the GitHub repository is public and anyone can access the source code from the link: https://github.com/michael-ngx/crypto-router. Our project is open for public contributions from open-source contributors.

3.12 Display of Routing Decision

After placing a market or limit order, the user can navigate to the Orders tab and select any trades to view a breakdown of how the routing algorithm has split the order across exchanges, along with the allocated amount for each venue (Figure 4b).

Summary and Conclusions

The Cryptocurrency Order Routing System successfully met its primary objective of providing a high-performance, open-source platform for optimizing trade execution across fragmented market venues. Throughout the design and implementation phases, the project team prioritized technical transparency and verifiable performance. Final system testing and verification confirmed that nearly all critical System Requirements Specifications (SyRS) were achieved, with the system delivering reliable liquidity aggregation and low-latency routing in a controlled, multi-user environment.

The final results of the project demonstrate high compliance with the established functional requirements and performance objectives. As detailed in the verification table, the system achieved an average end-to-end routing latency of 475 ms, significantly outperforming the original target of less than 2 seconds (R3). This success is attributed to the optimized C++ backend and the use of Single-Producer Single-Consumer (SPSC) ring buffers, which minimized the time required to move data from raw exchange feeds to the routing engine. Furthermore, the system achieved a 100% simulated market order fill rate (R4), validating the effectiveness of the custom execution engine in navigating live production order books. Market data aggregation also met its requirements, with all four exchanges delivering orderbook snapshots well within the 1-second update threshold, and stress testing confirmed stable throughput for up to 30 concurrent users, satisfying the multi-user accessibility and scalability objectives (R1, R2, O1).

The core design decisions were validated through testing. The monolithic C++ architecture eliminated inter-service communication overhead and maintained consistent snapshot freshness for routing decisions. The custom simulated execution engine, which fills orders against live production orderbook snapshots rather than stale sandbox data, proved to be a more representative testing environment than exchange-provided test endpoints, a finding motivated by early empirical testing that revealed sandbox orderbooks were too thin and artificial to produce meaningful slippage measurements. The post-trade analytics dashboard and stage-by-stage execution timeline further confirmed the value of the persistence model in providing users with a fully auditable and transparent view of how their orders were executed.

While the majority of objectives were met, the project did not successfully implement a standalone public API for external programmatic access (O3). This was a conscious design trade-off made during the final implementation phase to prioritize the stability of the core routing engine and the multi-user frontend experience. Despite this limitation, the system remains open for extension (O4) through its public GitHub repository, and users can interact with the full routing logic and analytics suite through the deployed web interface.

The work produced in this project has practical relevance beyond the academic context. Smart order routing is an active area in both traditional equities markets and the digital asset space, and the transparency goals of the system directly address documented gaps in retail cryptocurrency trading, where practices such as payment for order flow and undisclosed markups have attracted growing regulatory attention. For future work, the routing algorithm could be extended with more rigorous fee-adjusted cost optimization for limit orders, venue coverage could be expanded through the existing adapter architecture, and implementing the public API (O3) would complete the system's programmatic accessibility goal. A production-grade deployment would additionally require the lock-free data structure and thread pool improvements noted in the design documentation.

References

[1] K. Daugaard, "Writing a fast and versatile SPSC ring buffer," *Daugaard.org Blog*, 2022, https://tinyurl.com/3vwzk37s (accessed Mar. 21, 2026)

[2] "Top Cryptocurrency Spot Exchanges," *CoinMarketCap*, https://coinmarketcap.com/rankings/exchanges/ (accessed Mar. 21, 2026).

[3] A. Lehar, C. A. Parlour, and M. Zoican, "Fragmentation and optimal liquidity supply on decentralized exchanges," *SSRN Electronic Journal*, Nov. 2022. doi:10.2139/ssrn.4267429

[4] "Robinhood" *Robinhood*, https://robinhood.com/us/en/ (accessed Mar. 21, 2026).

[5] "Wealthsimple" *Wealthsimple*, https://www.wealthsimple.com/en-ca (accessed Mar. 21, 2026).

[6] "Florida AG probes robinhood crypto over claims of low-cost trading," *Reuters*, https://www.reuters.com/legal/government/florida-ag-probes-robinhood-crypto-over-claims-low-cost-trading-2025-07-10/ (accessed Mar. 21, 2026).

[7] "Robinhood investing review: 4 reasons to avoid this broker," *Wantfi*, https://wantfi.com/robinhood-stock-trading-app-review.html (accessed Mar. 21, 2026).

[8] "Understand why your cryptocurrency order filled at a different price," *Wealthsimple*, https://help.wealthsimple.com/hc/en-ca/articles/360060805574-Understand-why-your-cryptocurrency-order-filled-at-a-different-price (accessed Mar. 21, 2026).

[9] "Order Routing System (ORS)," *QuestDB Glossary*, https://questdb.com/glossary/order-routing-system-%28ors%29/ . (accessed Mar. 21, 2026).

[10] P. Powell and I. Smalley, "What is monolithic architecture?," IBM, https://www.ibm.com/think/topics/monolithic-architecture (accessed Mar. 21, 2026).

[11] T. Henker, J. Kusuma, and A. T. S. Raj, "Athena: Smart Order Routing on Centralized Crypto Exchanges Using a Unified Order Book," arXiv preprint arXiv:2404.02336, 2024, https://www.researchgate.net/publication/379309357_Athena_Smart_Order_Routing_on_Centralized_Crypto_Exchanges_using_a_Unified_Order_Book (accessed Mar. 21, 2026).

[12] P. Pujol and P. Brueckner, "Smart Order Routing and Best Execution," Semantics Scholar, 2019, https://www.semanticscholar.org/paper/Smart-Order-Routing-and-Best-Execution-Pujol-Brueckner/be6a9d7dfd3d4e95a7f7632f5c5181361bba5f8d (accessed Mar. 21, 2026).

[13] R. Almgren and N. Chriss, "Optimal Execution of Portfolio Transactions," *J. Risk*, Jan. 2001. doi: 10.21314/JOR.2001.041

[14] R. Cont and A. Kukanov, "Optimal order placement in limit order markets," *ArXiv*, Nov. 2014. doi:10.48550/arXiv.1210.1625

Appendices

Appendix A: System Requirements Specifications (SyRS)

ID	Specification	Description
R1	Deployed website with intuitive user-interface	**Requirement**: The platform shall be implemented and published as a website for easy and concurrent access by multiple users.
R2	Market data collection and display from ≥ 3 exchanges, updated every second	**Requirement**: System shall collect, aggregate, calculate, and display live market data from at least three exchanges with near real-time updates.
R3	Calculate and execute routing algorithm dynamically with < 2 seconds end-to-end delay	**Requirement**: Starting from user order placement in the interface, the system shall determine execution routes and connect to the execution simulation module to simulate execution on calculated exchanges, all within 2 seconds.
R4	100% market order fill rate, fill <1 second after user placement	**Requirement**: Market order shall be filled (simulated) within 1 second after placement on the platform by user.
R5	Limit order fill price must always be ≤ specified limit price	**Requirement**: Limit order shall be filled, or partially filled (simulated), at a price that does not exceed the specified limit price by user.
C1	Simulated trades	**Constraint**: The system must not execute real trades. Testing is limited to placing and filling orders on customized order fill simulator.
C2	Routing algorithm and data sources available for public access	**Constraint:** All source code underlying the platform shall be made public and have no access restrictions for verifiability. Data used for the routing calculation must be publicly available.
C3	Exchange and system connectivity via standard network procedure	**Constraint**: Must utilize standardized exchange APIs (REST and WebSocket).
O1	Maximize throughput	**Objective**: System shall be able to serve as many concurrent users and order requests as possible.
O2	Minimize latency	**Objective**: System shall minimize the calculation and execution time in R2, as well as time to complete order fills in R3.

O3	API access to system	**Objective**: The underlying execution system shall allow programmatic access to the public through APIs.
O4	Open for extension and public contributions	**Objective**: The code structure shall be built for scalability, and made public to accept open-source contributions.
O5	Display all decision-making data and calculations with analytics	**Objective**: The information in C3 shall be logged and displayed for users in an organized fashion, including options supporting data analytics.

Appendix B: Gantt Chart History

Fall

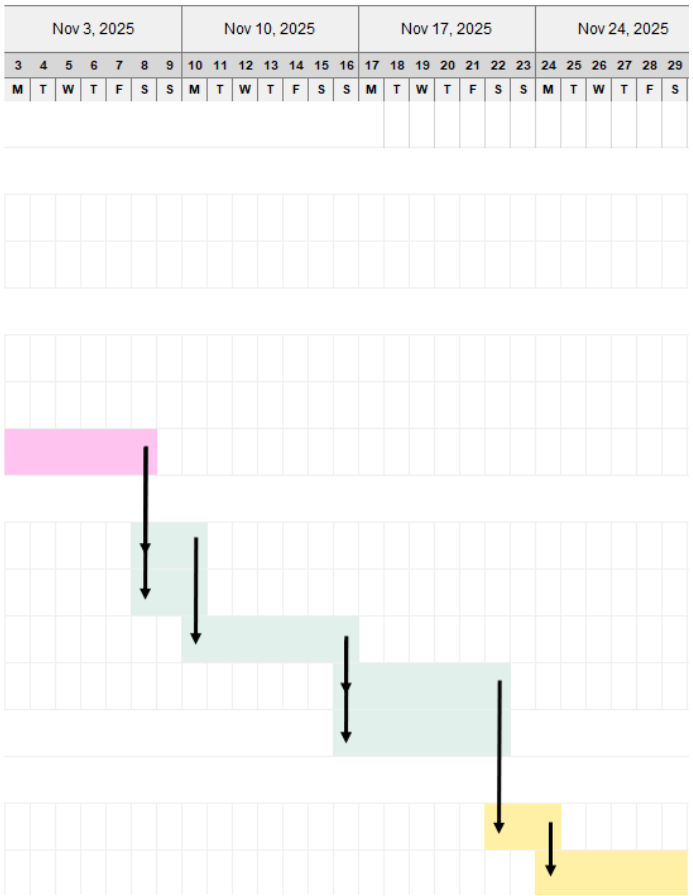
Order Routing System

ECE496

Project start: **Tue, 9/2/2025**

Display week: **10**

TASK	MEMBER	PROGRESS	START	END
Initiation				
Project Snapshot		100%	9/2/25	9/9/25
Preliminary Proposal		100%	9/9/25	9/18/25
Proposal and Implementation				
Detailed Proposal		100%	9/18/25	10/1/25
Updated Proposal		100%	10/1/25	10/18/25
Implementation Plan	All	100%	10/18/25	11/8/25
Partial Preliminary Implementation				
Exchange connected and live data retrieved	Michael	100%	11/8/25	11/10/25
Orderbook information obtained	Michael	100%	11/8/25	11/10/25
USD <-> BTC trading supported	Chris, Wen	100%	11/10/25	11/16/25
Clean functionality and UI completed	All	100%	11/16/25	11/22/25
Metrics and Logging implemented	Mindy	100%	11/16/25	11/22/25
Validate Implementation &				
Implementation Validated	All	100%	11/22/25	11/24/25
Preparation for demo	All	100%	11/24/25	11/29/25



Winter

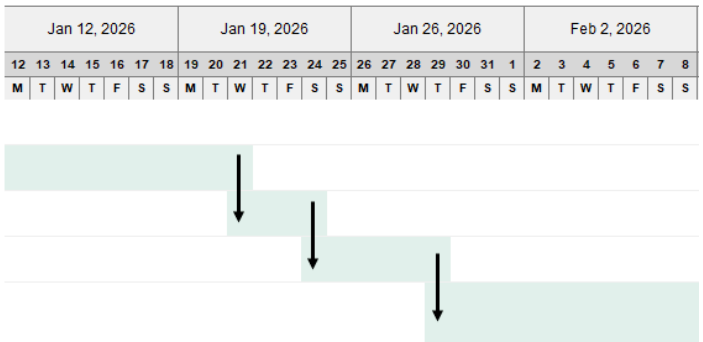
Order Routing System

ECE496

Project start: **Tue, 9/2/2025**

Display week: **20**

TASK	MEMBER	PROGRESS	START	END
Winter				
Implemented support for multiple exchanges	Michael, Chris	100%	1/11/26	1/21/26
Set up PostgresSQL database	Chris	100%	1/21/26	1/24/26
Orderbook analysis completed	Wen	100%	1/24/26	1/29/26
Implemented routing optimization with heuristics	Michael	100%	1/29/26	3/2/26



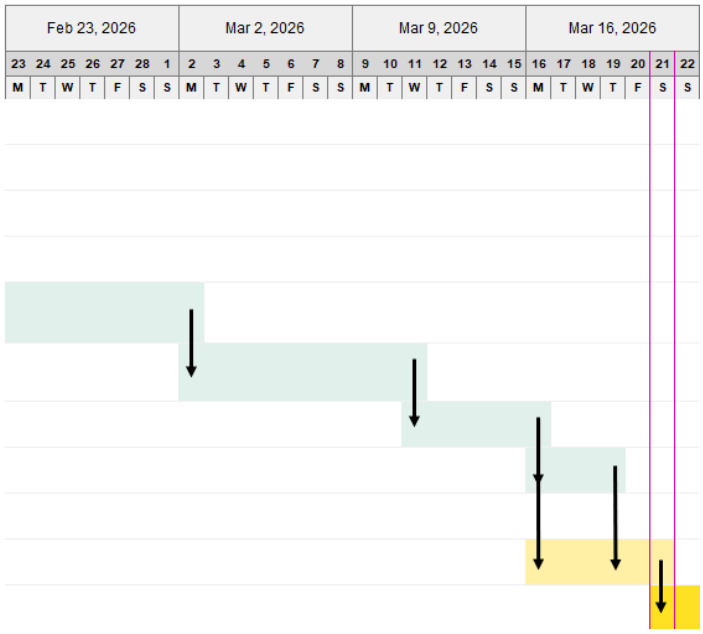
Order Routing System

ECE496

Tue, 9/2/2025

25

TASK	MEMBER	PROGRESS	START	END
Winter				
Implemented support for multiple exchanges	Michael, Chris	100%	1/11/26	1/21/26
Set up PostgresSQL database	Chris	100%	1/21/26	1/24/26
Orderbook analysis completed	Wen	100%	1/24/26	1/29/26
Implemented routing optimization with heuristics	Michael	100%	1/29/26	3/2/26
Implemented market and limit order functionality	Wen	100%	3/2/26	3/11/26
UI functionality updated	Chris, Mindy	100%	3/11/26	3/16/26
Performance Testing completed	Mindy	100%	3/16/26	3/19/26
Completion				
Final Report	All	100%	3/16/26	3/21/26
Presentation preparation for Design Fair	All	0%	3/21/26	3/27/26



Appendix C: Financial Plan

Consumables/Services						
Item	**Priority**	**Cost/unit**	**Quantity (# of hours)**	**Total Cost**	**Requires Funding**	

Cloud Hosting Service	1	$40/month	1 month	$40	y	
Github	1	Free	N/A	$0	n	
Crypto Exchange Market Data	4	Free	N/A	$0	n	
PostgresSQL Database on Superbase	1	Free	N/A	$0	n	
Poster	1	$60	N/A	$60	n	
Total Consumables/Services				$100		
Total Required Funding				$100		
Funding						
Students	$100					
Supervisor	$0					
Other	$0					
Requested from ECE496	$0					
Total Funding	**$100**					
Capital Equipment	(No capital equipment required for software project)					
Item	**Priority**	**Cost/unit**	**Quantity (# of hours)**	**Total Cost**	**Requires Funding**	**Kept/Paid for by**

						Students
Total Capital Equipment				**$0**		
Total Required Funding				**$0**		
Student Labour						
Item	Cost/unit	Quantity (# of hours)	Total Cost			
Viet Minh Nguyen	$35	137	$4795			
Somin Lee	$35	131	$4585			
Christopher Shih	$35	132	$4620			
Wen Ou	$35	132	$4620			
Total Student Labour (unfunded)			**$18620**			
Total Cost of Project			**$18720**			
Total Cost Requiring Funding			**$100**			

The main costs of the software projects include the cloud hosting service to host our crypto routing system and the poster for our final presentation. Market data can be retrieved using the exchange's API at no cost, and no hardware equipment is needed for the project. While our implementation plan initially allocated $300 for live trading capital, this amount was deemed insufficient to produce representative results given the capital scales ranging from thousands to tens of thousands of dollars targeted in our testing. As a result, we developed a custom simulated trading environment capable of more accurately reflecting real-world trading conditions at the intended scale.